ANRT REPORT

Technical Report

Module: Big Data Analytics

Major: Innovation and AMOA

Moroccan Business Intelligence Graph: A GraphRAG Platform for Arabic/French Sources

Prepared by:

Fatima Ezzahrae Fadel

Wijdane El Hariri

Under the supervision of

Hamid El Ghazi

Agence Nationale de Réglementation des Télécommunications

2026

Abstract

This report presents a Graph Retrieval-Augmented Generation (GraphRAG) platform for Moroccan business intelligence. The system crawls explicitly authorised Arabic, French and English sources, extracts HTML, RSS and PDF content with provenance, stores a raw zone in HDFS when available, and applies Spark-backed ETL before indexing documents and chunks. Mistral is used for entity and relation extraction, while NVIDIA embeddings support semantic retrieval. A FastAPI service exposes graph analytics, source exploration and grounded conversational answers, and a React frontend provides operational, source and evaluation views. The validation combines retrieval metrics with Ragas faithfulness, answer relevance and context-precision measurements, while clearly distinguishing smoke evaluation from independently reviewed test cases.

Keywords: GraphRAG, Moroccan business intelligence, Scrapy, PDF extraction, Mistral, NVIDIA embeddings, HDFS, Spark, FastAPI, Ragas.

Contents

General Introduction

Moroccan economic information is distributed across institutional websites, reports, RSS feeds and PDF publications. Analysts need answers that are fast to explore but also traceable to an original source. This report presents a Graph Retrieval-Augmented Generation (GraphRAG) platform prepared for **l'Agence Nationale de Réglementation des Télécommunications** (**ANRT**).

The objective is to combine respectful crawling, PDF and web extraction, multilingual processing, a knowledge graph, vector retrieval and a grounded chat assistant. The platform also exposes source provenance, pipeline status and evaluation statistics through a FastAPI backend and a React frontend.

The report first presents the architecture, then explains each principal pipeline component, before introducing Ragas and presenting the evaluation statistics and results. It closes with the main conclusions and perspectives.

Chapter 1

Architecture

1.1 Overview

The platform follows a layered architecture. The ingestion layer collects explicitly authorised sources. The processing layer validates, normalizes and chunks documents. The intelligence layer extracts entities and relations with Mistral and creates NVIDIA embeddings. The storage layer combines a raw zone, a graph store and a vector index. Finally, FastAPI exposes the services to a React interface.

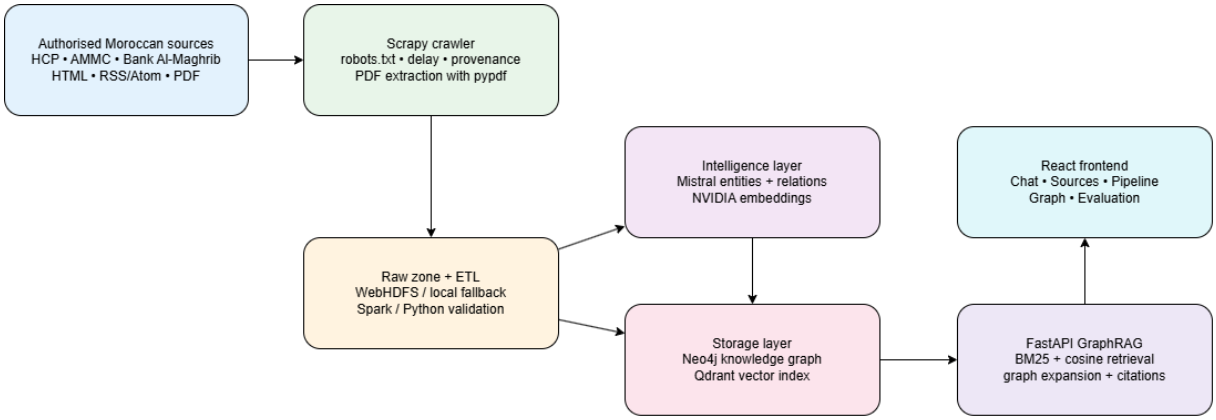


Figure 1.1: General architecture of the Moroccan GraphRAG platform.

1.2 Information Flow

The flow is: source URL → Scrapy item → raw document → Spark/Python ETL → chunks → graph and vector indexes → GraphRAG retrieval → cited answer. Each page keeps its URL, domain, title, language, document type and crawl timestamp.

1.3 Deployment

Docker Compose runs the API, frontend, ingestion worker, Qdrant and Neo4j. WebHDFS and Spark are optional integrations with local fallbacks for development. This keeps the prototype executable on a workstation while preserving a path to distributed deployment.

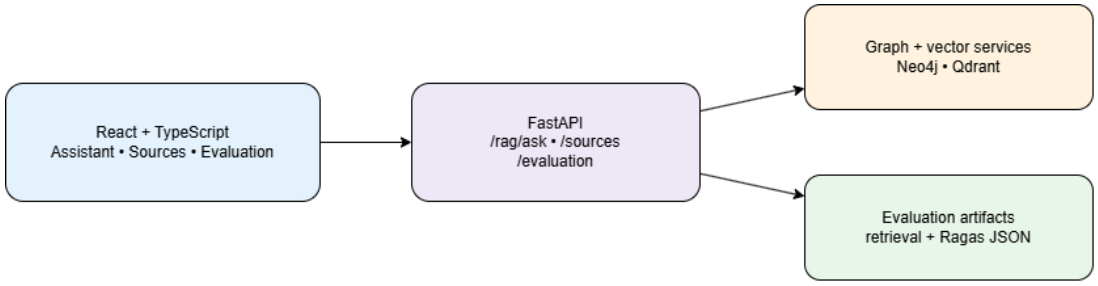


Figure 1.2: Frontend, API and service deployment view.

1.4 Current Corpus and Target Sources

The current indexed corpus contains **48 documents**: 35 HTML pages, 10 RSS/Atom entries and 3 PDF documents. The documents come from three domains: HCP (24), AMMC (16) and Bank Al-Maghrib (8). The configured targets are the HCP enterprise and economy pages, the AMMC issuer financial-statements page, Bank Al-Maghrib statistics, the Bank Al-Maghrib annual report PDF and the HCP RSS feed.

Chapter 2

Principal Pipeline Components

2.1 Crawler and PDF Sources

Scrapy crawls only configured and authorised targets, obeys robots.txt and uses conservative delays. HTML pages, RSS/Atom feeds and PDFs are dispatched through the same provenance-aware parser. PDF text is extracted with `pypdf`; short or failed extractions are rejected or recorded as failures.

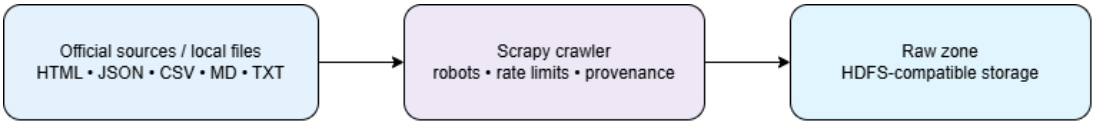


Figure 2.1: Source ingestion and crawler flow.

2.2 Raw Storage and ETL

The raw zone writes to WebHDFS when enabled and falls back to local storage when the namenode is unavailable. Spark performs deduplication, whitespace normalization, source-domain normalization and minimum-text validation. A pure-Python ETL path remains available for local testing.

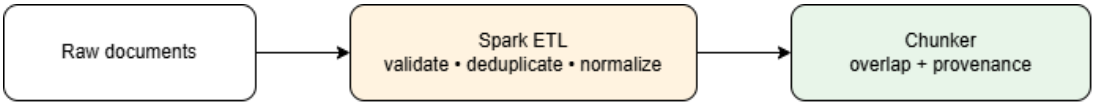


Figure 2.2: ETL, validation and chunking stages.

2.3 Chunking and Embeddings

Validated pages are divided into overlapping text chunks using an 800-character window and 120-character overlap. The resulting step is 680 characters, which preserves context across boundaries. NVIDIA's embedding model `nvidia/nemotron-3-embed-1b` indexes the chunks in Qdrant. The offline deterministic test fallback uses 256 dimensions; production Qdrant collections take the dimension returned by the NVIDIA endpoint.

2.4 BM25 and Hybrid Retrieval

Retrieval combines cosine similarity from embeddings with BM25 lexical relevance. BM25 uses $k_1 = 1.2$, $b = 0.75$ and a lexical multiplier of 2.2, with inverse document frequency computed over the indexed chunks and document-length normalization against the average chunk length. This hybrid score helps preserve exact names, acronyms and French/Arabic terms that may be missed by semantic similarity alone.

2.5 Entity and Relation Extraction

Mistral is used to extract typed entities and relations from chunks. The graph stores provenance through page and chunk links. A heuristic fallback, durable retry queue and extraction cache make failures visible without silently losing records.

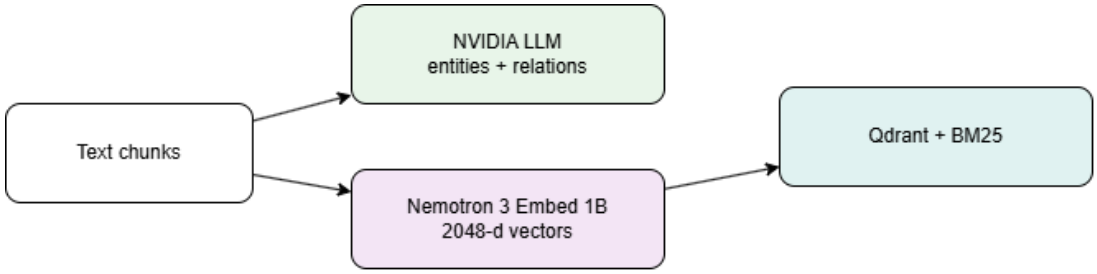


Figure 2.3: Entity extraction and embedding generation.

2.6 GraphRAG API and Frontend

FastAPI exposes grounded questions, graph analytics, source pagination, PDF viewing, pipeline status and evaluation statistics. The React frontend provides the assistant conversation, source pages, graph explorer, operational pipeline view and evaluation dashboard.

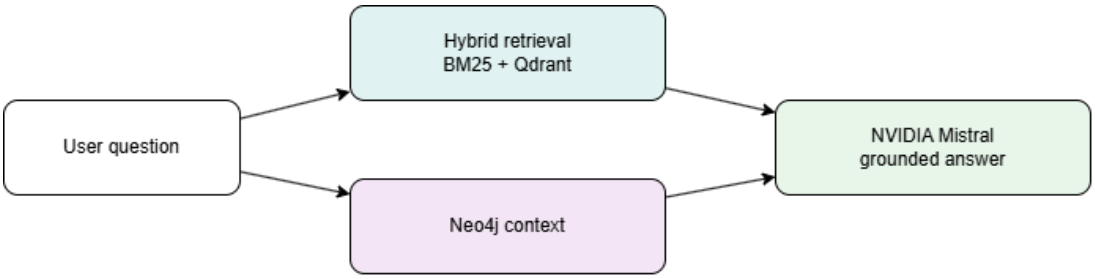


Figure 2.4: GraphRAG API and retrieval flow.

Chapter 3

Evaluation

3.1 What is Ragas?

Ragas is an evaluation framework for retrieval-augmented generation. It measures the quality of generated answers against their retrieved context. In this project, the principal metrics are faithfulness, answer relevancy and context precision. Faithfulness checks whether the answer is supported by the context; answer relevancy checks whether it addresses the question; and context precision checks whether the retrieved passages are useful and well-ranked.

3.2 Retrieval Evaluation

The current reviewed evaluation contains 39 documents, 534 chunks and 4 questions. Vector retrieval and GraphRAG both achieved Recall@5 of 0.75, MRR of 0.5833 and nDCG@5 of 0.625. Mean latency was 74.36 ms for vector retrieval and 94.32 ms for GraphRAG. The result is a tie: on this small corpus, the graph contributes explainability and provenance rather than a measured ranking advantage.

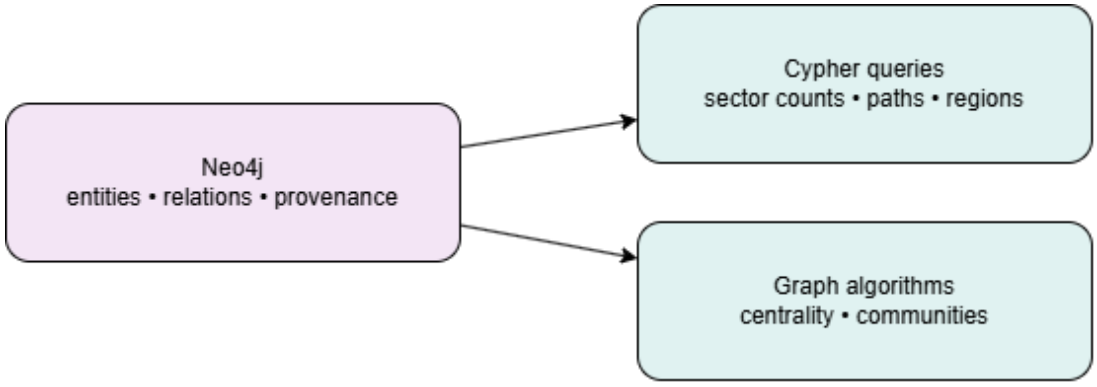


Figure 3.1: Knowledge-graph storage and analytics view.

3.3 Ragas Results

Ragas execution is wired through `scripts/evaluate_ragas.py` and its results are exposed by the FastAPI `/evaluation` endpoint. The frontend displays faithfulness, answer relevancy and context precision when `data/ragas_results.json` is available. Scores must be generated with valid provider credentials; missing scores are not presented as successful results.

3.4 Data Quality Statistics

The source-quality checker currently reports an average score of **76.6/100**. It combines authority, provenance, completeness and freshness. The corpus is therefore suitable for a functional demonstration, but the score is a provenance heuristic rather than a human-annotated truth-quality measure.

3.5 Interpretation and Limits

The retrieval cases are reviewed external cases, but the corpus remains small. Ragas scores should therefore be interpreted as an initial quality signal, not as a generalization guarantee. A larger reviewed Arabic/French question set and repeated provider-controlled runs are required for a final benchmark.

Conclusion and Perspectives

This report presented a Moroccan GraphRAG platform for source-grounded business intelligence at ANRT. The solution connects respectful HTML, RSS and PDF crawling with Spark/HDFS-ready ingestion, Mistral extraction, NVIDIA embeddings, graph and vector storage, FastAPI services and a React interface.

The platform was validated through automated tests, source-quality scoring, retrieval metrics and a Ragas execution path. The current retrieval benchmark reports a 0.75 Recall@5 tie between vector and GraphRAG retrieval, confirming a functional baseline while highlighting the need for a larger reviewed evaluation set.

Future work includes adding an HDFS namenode and Spark cluster to the deployment, running Ragas with a curated multilingual dataset, adding authentication and authorization, and measuring extraction quality separately from retrieval quality.